how-to guide Arduino

Selbstlernkonzept
Arduino-Challenges

dein Technik Lehrer

Dieser how-to guide gehört zum Selbstlernkonzept **„Arduino-Challenges“**. Er hilft dir, wenn du bei einer Challenge nicht weiterkommst. Du musst ihn nicht von vorne bis hinten lesen – schlag einfach die Seite nach, die auf deiner Challenge-Karte steht.

Alle Challenges gehen mit **Blöcken** und mit **Text**. Deshalb siehst du bei den Befehlen immer beides.

Version 10.2026
Daniel Beyrodt

Nutzungsbedingungen:
Dieses Werk ist unter einer Creative-Commons-Lizenz vom Typ Namensnennung – Nicht-kommerziell – Weitergabe unter gleichen Bedingungen 4.0 International zugänglich.
https://creativecommons.org/licenses/by-nc-sa/4.0/deed.de
Ausgenommen sind Screenshots und Abbildungen Dritter (z. B. Tinkercad/Autodesk, Fritzing); für sie gelten die Rechte der jeweiligen Inhaber.

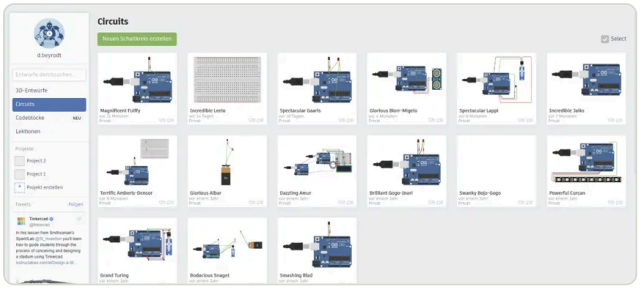
Inhalt

Tinkercad starten	4
Code-Editor und Simulation	5
Das Steckbrett	6
Die LED richtig anschließen	7
Ausgänge schalten und warten	8
Wiederholen: Schleifen	9
Variablen und Rechnen	10
Dimmen mit PWM	11
Die RGB-LED	12
Der serielle Monitor	13
Analoge Sensoren: der Fotowiderstand	14
Wenn – dann – sonst	15
Der Taster	16
Der Servo	17
Töne mit dem Piezo	18
Der Ultraschallsensor HC-SR04	19
Der Temperatursensor TMP36	20
Gassensor und Bodenfeuchtesensor	21
Das Relais	22
Echt aufbauen	23
Programm auf den Arduino laden	24
Fehlerarten	25
Fehler beheben	26

Tinkercad starten

Anmelden

Öffne **www.tinkercad.com** und klicke auf **„Anmelden“**. Hast du von deiner Lehrkraft einen Klassencode bekommen, wähle **„Schüler mit Klassencode“**. Gib zuerst den Klassencode ein, dann deinen Spitznamen.



Neuen Schaltkreis anlegen

Klicke im Dashboard links auf **„Circuits“** und dann auf **„Neuen Schaltkreis erstellen“**. Gib deinem Schaltkreis einen Namen, z. B. „Challenge 01“.

Bauteile holen

Rechts stehen die **Komponenten**. Über das Suchfeld findest du jedes Bauteil (z. B. „LED“). Ziehe es auf die Arbeitsfläche. Unter **„Starter → Arduino“** gibt es fertige Vorlagen, z. B. Arduino mit Steckbrett.

Eigenschaften ändern

Klickst du ein Bauteil an, öffnet sich der **Inspektor**. Dort stellst du z. B. den Widerstandswert (330 Ω) oder die LED-Farbe ein.

Daniel Beyrodt · CC BY-NC-SA 4.0

Code-Editor und Simulation

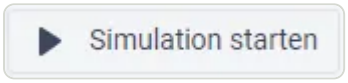


Code öffnen

Klicke oben rechts auf **„Code“**. Im Auswahlfeld wählst du: **Blöcke**, **Blöcke + Text** oder **Text**. Alle Challenges gehen mit Blöcken und mit Text.

Wichtig beim Umschalten

Von **Blöcke** zu **Text** geht es ohne Verlust. Wechselst du von **Text** zurück zu **Blöcken**, wird dein Text-Code gelöscht.



Simulation starten

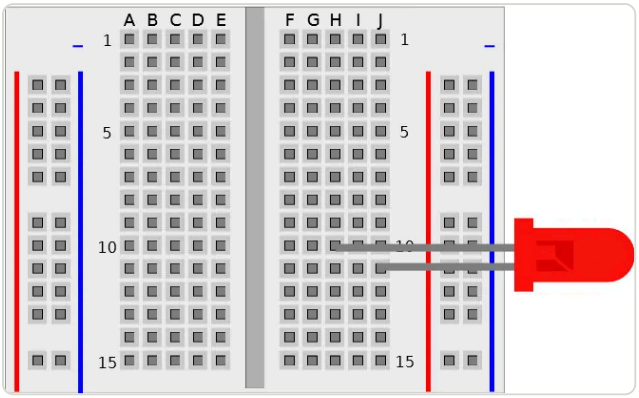
Mit **„Simulation starten“** läuft dein Programm. Während der Simulation kannst du Sensoren anklicken und einstellen (z. B. Helligkeit, Abstand, Temperatur). Unten im Code-Bereich öffnest du den **Serien-Monitor**.

Blöcke + Text

In der Ansicht „Blöcke + Text“ siehst du, welchen C++-Code Tinkercad aus deinen Blöcken macht. Diesen Text kopierst du, wenn du dein Programm auf den echten Arduino laden willst.

Daniel Beyrodt · CC BY-NC-SA 4.0

Das Steckbrett



Welche Löcher sind verbunden?

Jede **nummerierte Reihe** verbindet ihre Löcher – aber nur **a bis e** und getrennt davon **f bis j**. Die **Mittelrinne** trennt beide Hälften.

Die Schienen

Die Schienen am Rand (**+** rot, **–** schwarz/blau) verbinden alle Löcher einer Zeile. Verbinde **5V** mit der +-Schiene und **GND** mit der –-Schiene. Obere und untere Schienen sind **nicht** miteinander verbunden.

Tipp in Tinkercad

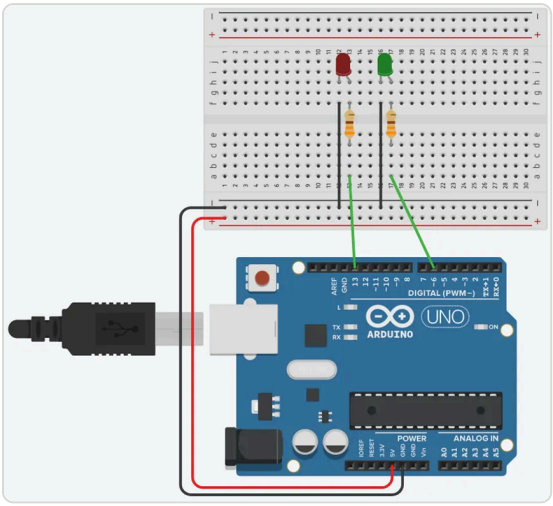
Fährst du mit der Maus über ein Loch, zeigt Tinkercad, welche Löcher damit verbunden sind.

Häufiger Fehler

Stecken beide Beine eines Bauteils in **derselben Reihe**, sind sie kurzgeschlossen – das Bauteil funktioniert nicht.

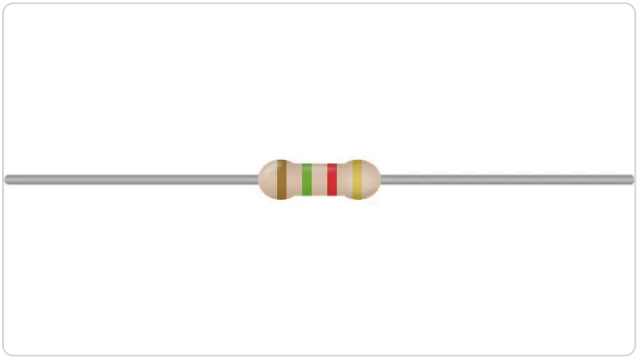
Daniel Beyrodt · CC BY-NC-SA 4.0

Die LED richtig anschließen



Polung

Langes Bein = Anode (+) → Richtung Arduino-Pin. **Kurzes Bein = Kathode (–)** → Richtung GND. In Tinkercad ist die Anode das **geknickte** Bein.



Vorwiderstand

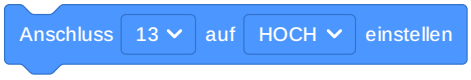
Ein Pin liefert 5 V, eine LED verträgt nur etwa 2 V. Ein **Vorwiderstand** von **220 Ω oder 330 Ω** in Reihe schützt sie. Jede LED bekommt einen eigenen Widerstand.

Farbringe

330 Ω = **orange – orange – braun** (– gold). 220 Ω = **rot – rot – braun**. 10 kΩ = **braun – schwarz – orange**.

Daniel Beyrodt · CC BY-NC-SA 4.0

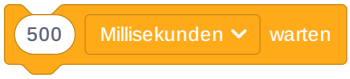
Ausgänge schalten und warten




```
digitalWrite(13, HIGH);
```

Pin einschalten

HOCH (HIGH) bedeutet 5 V am Pin – die LED leuchtet. NIEDRIG (LOW) bedeutet 0 V – die LED ist aus. In Blöcken: **Ausgang → „Anschluss … auf … einstellen“**.




```
delay(500);
```

Warten

Steuerung → „… warten“. Du kannst Sekunden oder Millisekunden wählen. 1 Sekunde = 1000 Millisekunden.

---

setup und loop

Im Text-Code steht in **setup()**, was einmal beim Start passiert (z. B. `pinMode(13, OUTPUT);`). **loop()** läuft danach immer wieder – bei Blöcken ist das „für immer“.

Text-Regeln

Jeder Befehl endet mit einem **Semikolon ;**. Geschweifte Klammern **{ }** öffnen und schließen einen Abschnitt (Tastatur: AltGr + 7 / AltGr + 0).

Daniel Beyrodt · CC BY-NC-SA 4.0

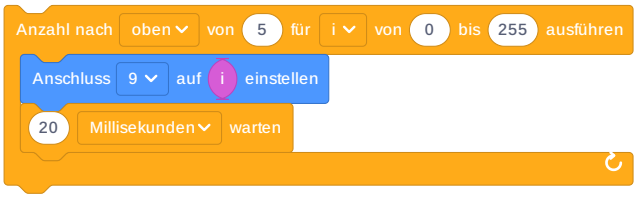
Wiederholen: Schleifen




```
for (int i = 1; i <= 3; i++) {
  digitalWrite(13, HIGH);
  delay(200);
  digitalWrite(13, LOW);
  delay(200);
}
```

Einfach wiederholen

Steuerung → „... Mal wiederholen“. Alles, was im Block steckt, läuft so oft wie angegeben.




```
for (int i = 0; i <= 255; i += 5) {
  analogWrite(9, i);
  delay(20);
}
```

Zählen mit Variable

„Anzahl nach oben von ... für i von ... bis ... ausführen“ zählt eine Variable hoch oder runter. Die Variable (hier **i**) kannst du im Inneren benutzen, z. B. als Helligkeit oder Winkel.

Achtung Endlosschleife

Ändert sich der Zähler nie so, dass die Bedingung falsch wird, hört die Schleife nie auf. Das Programm hängt dann fest.

Daniel Beyrodt · CC BY-NC-SA 4.0

Variablen und Rechnen

Was ist eine Variable?

Eine Variable ist ein **Speicherplatz mit Namen** für eine Zahl. In Blöcken: **Variablen → „Variable erstellen“**. Namen ohne Leerzeichen und ohne Umlaute.




```
zeit = 600;
zeit = zeit - 50;
```

Wert setzen und ändern

„... auf ... einstellen“ setzt einen Wert. „... um ... ändern“ zählt etwas dazu (mit Minus: weniger).




```
analogWrite(9, 255 - i);
```

Mit Operatoren rechnen

Unter **Mathematik** findest du + – × ÷. Du kannst Variablen und Zahlen verrechnen und das Ergebnis direkt in einen Block einsetzen.

Im Text

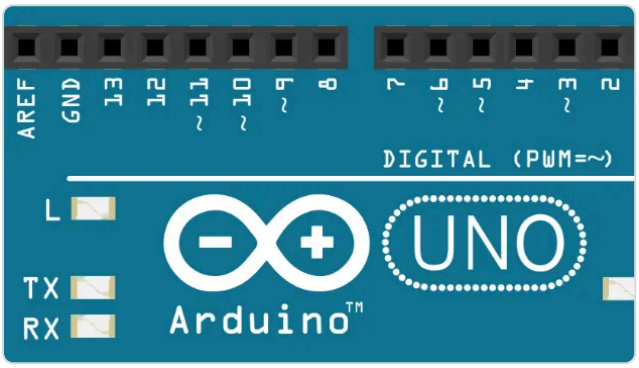
Eine Variable wird vor **setup()** angelegt: `int zeit = 0;`. „int“ bedeutet ganze Zahl.

Daniel Beyrodt · CC BY-NC-SA 4.0

Dimmen mit PWM

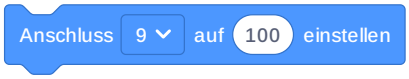
Der Trick

Der Arduino schaltet den Pin sehr schnell an und aus (**Pulsweitenmodulation**). Je länger die An-Zeit, desto heller die LED. Werte von **0** (aus) bis **255** (ganz hell).



Nur an ~Pins

PWM geht nur an den Pins mit Welle **~**: **3, 5, 6, 9, 10, 11**.



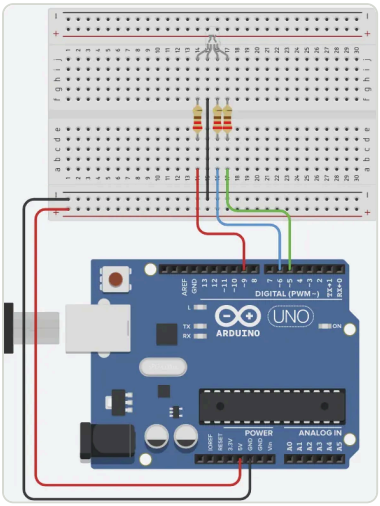

```
analogWrite(9, 100);
```

Der Block

Im Block „Anschluss … auf … einstellen“ trägst du statt HOCH eine Zahl ein.

Daniel Beyrodt · CC BY-NC-SA 4.0

Die RGB-LED



Aufbau

Eine RGB-LED enthält eine rote, eine grüne und eine blaue LED. Das **längste Bein** ist der gemeinsame Minuspol (GND). Jede Farbe braucht einen eigenen Widerstand und einen **~Pin**. In unseren Challenges: **Rot ~9, Grün ~5, Blau ~6**.




```
analogWrite(9, 255);
analogWrite(5, 255);
analogWrite(6, 0);
```

Farbe mit einem Block

Ausgang → „RGB-LED an Anschlüssen … auf Farbe … einstellen“. Wähle die Pins in der Reihenfolge Rot, Grün, Blau und klicke auf den Farbkreis.




```
analogWrite(9, 255);
analogWrite(5, 255);
analogWrite(6, 0);
```

Farbe im Text

Im Text setzt du jede Farbe einzeln mit `analogWrite`. Gelb = Rot 255 + Grün 255 + Blau 0.

Farben mischen

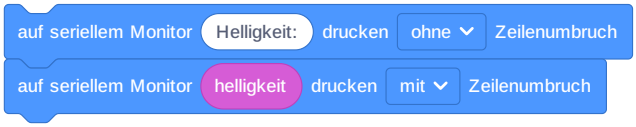
Rot + Grün = **Gelb** · Rot + Blau = **Lila** · Grün + Blau = **Türkis** · alle drei = **Weiß**.

Daniel Beyrodt · CC BY-NC-SA 4.0

Der serielle Monitor

Wozu?

Der Arduino kann Zahlen und Text an den Computer schicken. So siehst du Messwerte und findest Fehler. In Tinkercad heißt das Fenster **„Serien-Monitor“** (unten im Code-Bereich).




```
Serial.print("Helligkeit: ");
Serial.println(helligkeit);
```

Der Block

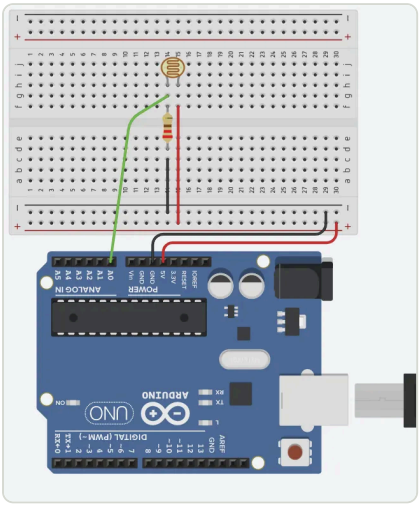
Ausgang → „auf seriellem Monitor … drucken … Zeilenumbruch“. „ohne“ schreibt in derselben Zeile weiter, „mit“ beginnt danach eine neue Zeile.

Im Text

In **setup()** muss `Serial.begin(9600);` stehen. Bei Blöcken erledigt Tinkercad das automatisch.

Daniel Beyrodt · CC BY-NC-SA 4.0

Analoge Sensoren: der Fotowiderstand



Spannungsteiler

Der Fotowiderstand kommt mit einem **10-kΩ-Widerstand** in Reihe zwischen 5V und GND. Der Punkt dazwischen geht an **A0**. Je heller es ist, desto größer wird hier der Messwert (0 bis 1023).




```
helligkeit = analogRead(A0);
```

Auslesen

Eingang → „Analogen Anschluss … lesen“. Speichere den Wert in einer Variablen.

In Tinkercad testen

Klicke während der Simulation auf den Fotowiderstand und verschiebe den Regler. Beobachte die Werte im Serien-Monitor.

Daniel Beyrodt · CC BY-NC-SA 4.0

Wenn – dann – sonst



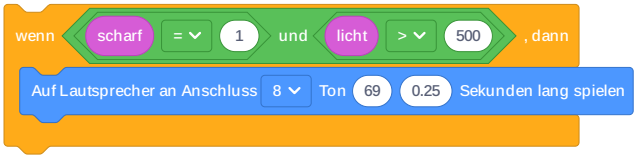

```
if (helligkeit < 300) {
  digitalWrite(13, HIGH);
} else {
  digitalWrite(13, LOW);
}
```

Bedingung

Steuerung → „wenn … dann“ führt Blöcke nur aus, wenn die Bedingung stimmt. **„wenn … dann … sonst“** hat zusätzlich einen Teil für „stimmt nicht“.

Vergleiche

Unter **Mathematik**: **<** kleiner, **>** größer, **≤ / ≥** kleiner/größer oder gleich, **=** gleich (im Text: ==), **≠** ungleich (!=).




```
if (scharf == 1 && licht > 500) {
  tone(8, 880, 250);
}
```

und / oder

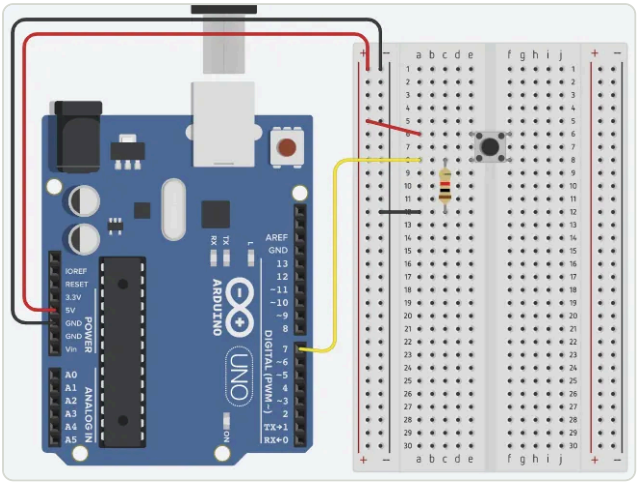
Mit **„und“** müssen beide Bedingungen stimmen, mit **„oder“** reicht eine. Im Text: && und ||.

Mehr als zwei Fälle

Lege in den „sonst“-Teil einen weiteren „wenn … dann … sonst“-Block. Im Text schreibst du `else if`.

Daniel Beyrodt · CC BY-NC-SA 4.0

Der Taster



Anschluss mit Pulldown

Ein Bein des Tasters an **5V**, das gegenüberliegende an den **Pin** (z. B. 7). Vom Pin geht ein **Widerstand (z. B. 10 kΩ)** nach **GND**. Er „zieht“ den Pin auf 0 V, solange der Taster nicht gedrückt ist.




```
taster = digitalRead(7);
```

Auslesen

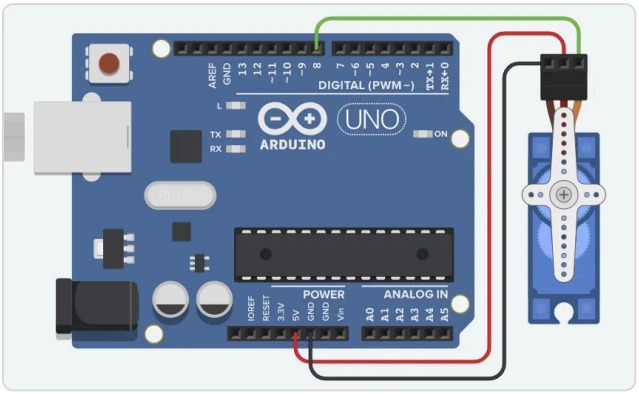
Eingang → „Digitalen Anschluss … lesen“ liefert HOCH (gedrückt) oder NIEDRIG (nicht gedrückt).

Im Text

In **setup()**: `pinMode(7, INPUT);`. Vergleich mit ==: `if (taster == HIGH)`.

Daniel Beyrodt · CC BY-NC-SA 4.0

Der Servo



Anschluss

Braun = **GND**, Rot = **5V**, Orange = **Signal** an einen digitalen Pin (z. B. 8). Ein Servo dreht sich auf einen Winkel von 0° bis 180°.




```
#include <Servo.h>
Servo servo_8;
void setup() { servo_8.attach(8); }
void loop() {
  servo_8.write(90);
}
```

Der Block

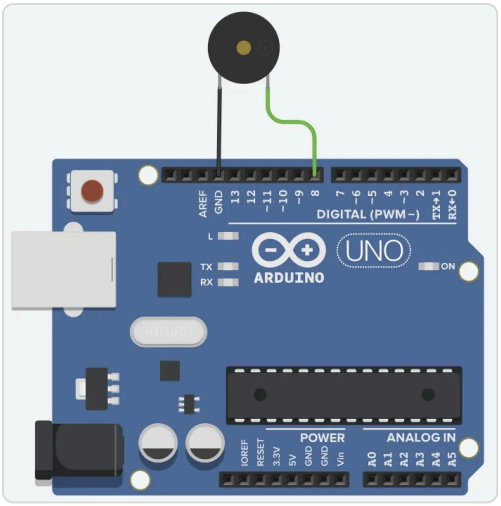
Ausgang → „Servo an Anschluss … auf … Grad drehen“. Als Winkel kannst du auch eine Variable einsetzen.

Im Text

`#include <Servo.h>` ganz oben, dann `Servo schranke;`. In **setup()**: `schranke.attach(8);`. Winkel setzen: `schranke.write(90);`

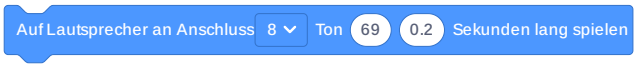
Daniel Beyrodt · CC BY-NC-SA 4.0

Töne mit dem Piezo



Anschluss

Ein **passiver** Piezo-Lautsprecher kommt mit **+** an einen Pin (z. B. 8) und mit **–** an GND. Er kann verschiedene Tonhöhen spielen.




```
tone(8, 880, 200);
```

Der Block

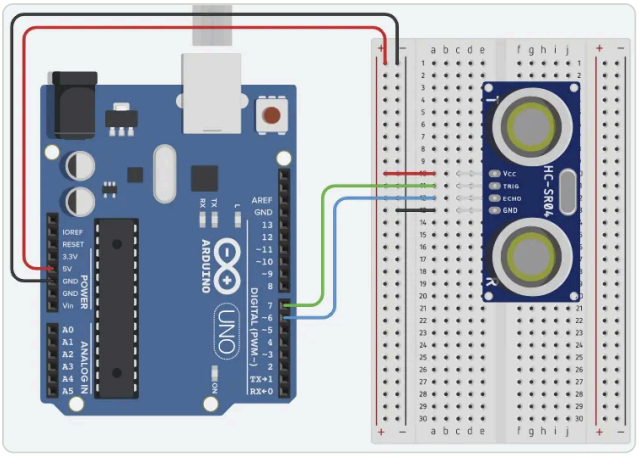
Ausgang → „Auf Lautsprecher an Anschluss … Ton … … Sekunden lang spielen“. Die Zahl bei „Ton“ ist eine **Notennummer**: 57 = A (440 Hz), 60 = C (523 Hz), 69 = A (880 Hz). +12 ist eine Oktave höher.

Im Text

`tone(Pin, Frequenz in Hz, Dauer in ms);` spielt einen Ton, `noTone(Pin);` stoppt ihn. Tinkercad rechnet die Notennummer aus dem Block in Hz um.

Daniel Beyrodt · CC BY-NC-SA 4.0

Der Ultraschallsensor HC-SR04



Anschluss

VCC an 5V, **GND** an GND, **Trig** an Pin 7, **Echo** an Pin 6. Der Sensor schickt einen Ultraschall-Impuls los und misst, wann das Echo zurückkommt.




```
abstand = 0.01723 *
readUltrasonicDistance(7, 6);
```

Der Block

Eingang → „Ultraschall-Abstandssensor an Trigger-Anschluss … Echo-Anschluss … in Einheiten cm“. Speichere den Abstand in einer Variablen.

Im Text

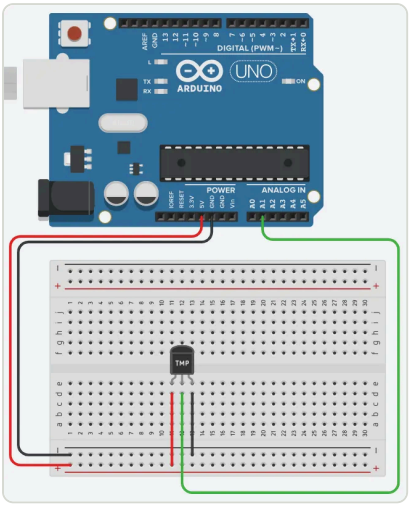
Tinkercad erzeugt dafür eine eigene Funktion `readUltrasonicDistance(trig, echo)`. Der Abstand in cm ist `0.01723 * readUltrasonicDistance(7, 6)`. Kopiere die Funktion aus „Blöcke + Text“ mit.

In Tinkercad testen

Klicke während der Simulation auf den Sensor und ziehe den Punkt (das Hindernis) näher heran oder weiter weg.

Daniel Beyrodt · CC BY-NC-SA 4.0

Der Temperatursensor TMP36



Anschluss

Halte die **flache Seite** zu dir: **links 5V**, **Mitte Signal** (z. B. A1), **rechts GND**. Vertauschst du 5V und GND, wird der Sensor heiß – sofort abziehen!




```
temperatur = (-40 + 0.488155 *
(analogRead(A1) - 20));
```

Der Block

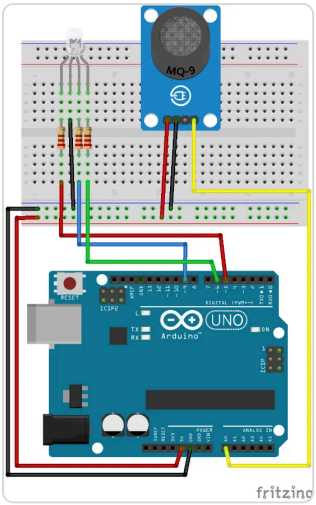
Eingang → „Temperatursensor an Anschluss … in Einheiten °C lesen“.

Im Text

Tinkercad rechnet den Messwert mit einer Formel in °C um: `temperatur = (-40 + 0.488155 * (analogRead(A1) - 20));`

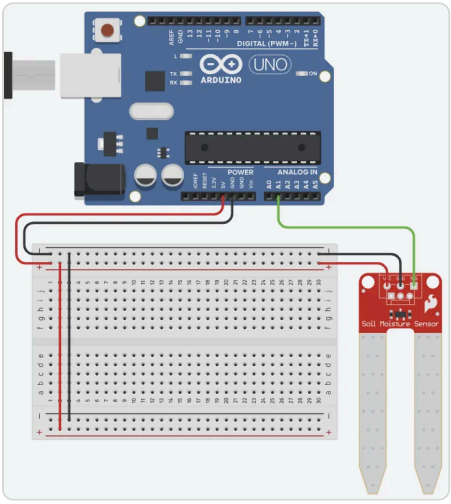
Daniel Beyrodt · CC BY-NC-SA 4.0

Gassensor und Bodenfeuchtesensor



Gassensor

Der Gassensor misst, wie belastet die Luft ist. In Tinkercad heißt er „Gassensor“. Echt nimmst du z. B. einen **MQ-135 auf einem Breakout-Board**: VCC an 5V, GND an GND, **AO** an A0. Der echte Sensor braucht nach dem Einschalten einige Minuten zum Aufheizen.



Bodenfeuchtesensor

+ an 5V, **–** an GND, **Signal** an einen analogen Pin (z. B. A1). In Tinkercad wird der Wert größer, je nasser die Erde ist. **Bei manchen echten Sensoren ist es umgekehrt** – miss deshalb immer zuerst trocken und nass.




```
luft = analogRead(A0);
```

Auslesen

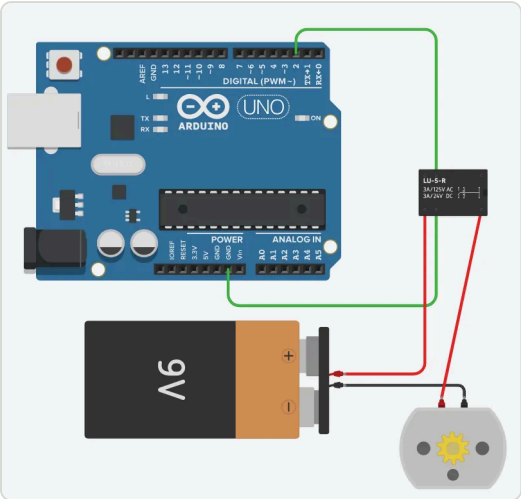
Beide liest du wie den Fotowiderstand mit „Analogen Anschluss … lesen“ aus.

Grenzwerte finden

Lass dir die Werte im Serien-Monitor anzeigen und notiere sie für die verschiedenen Zustände. Lege deinen Grenzwert **zwischen** die Werte.

Daniel Beyrodt · CC BY-NC-SA 4.0

Das Relais



Was macht ein Relais?

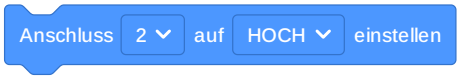
Ein Relais ist ein **Schalter**, den der Arduino mit einem kleinen Steuerstrom umlegt. So schaltet er Geräte mit eigener Stromquelle, z. B. eine Pumpe mit Batterie.

Anschluss

Relais-Modul: **VCC** an 5V, **GND** an GND, **IN** an einen Pin (z. B. 2). Der Verbraucher und seine Batterie hängen am **Schaltkontakt** des Relais – nicht am Arduino.

Sicherheit

Niemals 230 V! Arbeite nur mit Batterien oder Netzteilen mit Schutzkleinspannung.




```
digitalWrite(2, HIGH);
```

Schalten

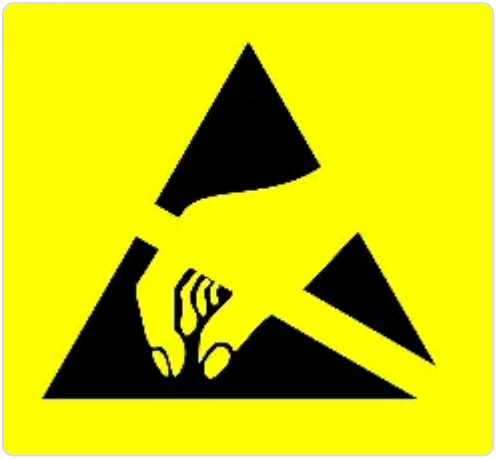
Pin auf HOCH = Relais zieht an (Gerät an), NIEDRIG = Gerät aus. **Achtung:** Manche echten Relais-Module schalten schon bei NIEDRIG ein („Low Level Trigger“). Dann drehst du HOCH und NIEDRIG im Programm um.

Daniel Beyrodt · CC BY-NC-SA 4.0

Echt aufbauen

Erst die Simulation

Baue jede Challenge **zuerst in Tinkercad** und teste sie. Erst wenn alles funktioniert, baust du sie echt auf. Nimm dein Tinkercad-Bild als Vorlage.



Sicher arbeiten

Baue nur um, wenn das **USB-Kabel abgezogen** ist. Verbinde **nie 5V direkt mit GND** (Kurzschluss). Schließe den Arduino nie gleichzeitig per USB und Batterie an. Fasse vorher kurz die Heizung an, um dich zu entladen (ESD).

Sorgfältig stecken

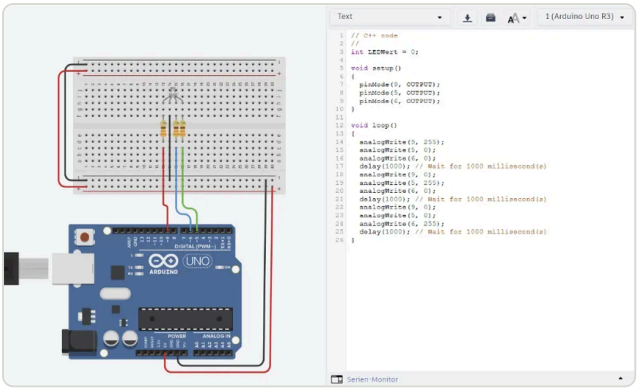
Drücke Bauteile und Kabel ganz ins Steckbrett. Prüfe jede Verbindung mit deinem Tinkercad-Bild: gleiche Reihe? richtige Schiene? richtiger Pin?

Farben nutzen

Nimm **rote** Kabel für 5V und **schwarze** für GND. Dann findest du Fehler schneller.

Daniel Beyrodt · CC BY-NC-SA 4.0

Programm auf den Arduino laden



Code aus Tinkercad holen

Stelle in Tinkercad auf **„Blöcke + Text“** oder **„Text“**. Markiere den Text (Strg + A) und kopiere ihn (Strg + C). Du kannst ihn auch mit dem Download-Knopf als Datei speichern.

Arduino IDE vorbereiten

Schließe den Arduino per USB an und öffne die **Arduino IDE**. Füge den Code ein (Strg + V). Wähle **Werkzeuge** → **Board** → **Arduino Uno** und unter **Werkzeuge** → **Port** den Port des Arduino.



Hochladen

Klicke auf den **Pfeil nach rechts** (Hochladen). Unten erscheint **„Hochladen abgeschlossen“**. Jetzt läuft das Programm auf dem echten Arduino.

Daniel Beyrodt · CC BY-NC-SA 4.0

Fehlerarten

Fehler im Text-Code

Fehlt ein **Semikolon** oder eine **Klammer**, startet die Simulation nicht. Tinkercad und die Arduino IDE zeigen die Zeile mit dem Fehler an – der Fehler steht oft eine Zeile **darüber**.

Fehler in der Schaltung

LED falsch herum · beide Beine in derselben Reihe · Kabel eine Reihe daneben · Widerstand fehlt · Schiene nicht mit 5V/GND verbunden.

Fehler in der Logik

Das Programm läuft, macht aber etwas anderes: falscher Pin · Pause fehlt (LED scheint immer an) · Bedingung falsch herum (< statt >) · Grenzwert passt nicht zu den Messwerten.

Fehler beim echten Arduino

Hochladen klappt nicht (falsches Board oder falscher Port) · USB-Kabel ist nur ein Ladekabel · Wackelkontakt · Bauteil defekt.

Daniel Beyrodt · CC BY-NC-SA 4.0

Fehler beheben

1. Schaltung prüfen

Vergleiche Kabel für Kabel mit der Vorlage. Fahre in Tinkercad mit der Maus über die Löcher. Prüfe die Polung der LEDs.

---

2. Pins vergleichen

Stimmen die Pins im Programm mit den Pins in der Schaltung überein?

---

3. Werte anzeigen

Lass dir Messwerte und Variablen im **Serien-Monitor** anzeigen. So siehst du, was der Arduino „denkt“.

---

4. Schritt für Schritt

Teste Teile einzeln: erst eine LED, dann die nächste. Baue das Programm Block für Block auf und teste zwischendurch.

---

5. Hilfe holen

Frag zuerst deine Nachbarin oder deinen Nachbarn, dann deine Lehrkraft. In der Arduino-Lernplattform findest du alles noch ausführlicher.

Daniel Beyrodt · CC BY-NC-SA 4.0